

QCM Expert — Processus et Threads

Q1. Dans le modèle du processus, lequel des éléments suivants appartient au **contexte d'exécution** et non au programme binaire ?

- A. Instructions exécutables
- B. Données statiques
- C. Registres du processeur
- D. Segment TEXT

Q2. Le segment **BSS** d'un processus contient :

- A. Les constantes symboliques
- B. Les variables locales
- C. Les variables globales non initialisées
- D. Le code exécutable

Q3. Lors du chargement d'un programme en mémoire, le rôle principal du chargeur est :

- A. Compiler le programme
- B. Allouer uniquement le segment TEXT
- C. Copier le programme disque vers la mémoire physique
- D. Traduire les adresses physiques en logiques

Q4. La mémoire physique est définie comme :

- A. Un espace logique propre à chaque processus
- B. Un ensemble de pages virtuelles
- C. La mémoire installée vue comme un tableau d'octets
- D. Une abstraction logicielle uniquement

Q5. La mémoire virtuelle donne à chaque processus l'illusion :

- A. D'un CPU dédié
- B. D'un espace mémoire privé, contigu et étendu
- C. D'une mémoire strictement physique
- D. D'une exécution séquentielle

Q6. Les **pages virtuelles** sont :

- A. Des registres CPU
- B. Des blocs de taille fixe du programme
- C. Des segments physiques
- D. Des threads légers

Q7. Les cases en mémoire réelle sont appelées :

- A. Pages virtuelles
- B. Segments
- C. Frames physiques
- D. Blocs logiques

Q8. Dans l'exemple d'adressage, l'adresse virtuelle est obtenue par :

- A. Adresse physique + offset
- B. Adresse logique - offset
- C. Adresse virtuelle de base + décalage
- D. Adresse disque + table des pages linéaire

Q9. La conversion de 100 (base 10) utilisée dans l'exemple donne :

- A. 0x100
- B. 0x64
- C. 0x32
- D. 0xA0

Q10. Si l'AVB vaut 0x400000, l'adresse virtuelle finale est :

- A. 0x400100
- B. 0x400064
- C. 0x400032
- D. 0x400200

Q11. Un processus à l'état **Bloqué** :

- A. Utilise le CPU
- B. Attend un événement externe
- C. Est prêt à s'exécuter
- D. Est terminé

Q12. Le passage entre les états **Prêt** et **En cours** est géré par :

- A. Le chargeur
- B. Le swapper
- C. L'ordonnanceur
- D. Le compilateur

Q13. Le PCB contient toutes les informations suivantes, sauf :

- A. PID
- B. Valeurs des registres
- C. Code source
- D. État du processus

Q14. L'ordonnancement **long terme** décide :

- A. Quel processus obtient le CPU
- B. Quels processus entrent en mémoire
- C. La durée du quantum
- D. L'ordre des interruptions

Q15. L'ordonnanceur à court terme est aussi appelé :

- A. Swapper
- B. Dispatcher
- C. Chargeur
- D. Moniteur

Q16. Un algorithme **préemptif** :

- A. Attend la fin du processus courant
- B. Interrompt un processus en cours
- C. Ignore la priorité
- D. Est toujours FCFS

Q17. Une **condition de course** survient lorsque :

- A. Les processus sont synchronisés
- B. L'ordre d'exécution influence le résultat
- C. Un seul processus s'exécute
- D. La mémoire est virtuelle

Q18. La section critique correspond :

- A. Au code du noyau
- B. Au code accédant à une ressource partagée
- C. À la pile du processus
- D. À la table des pages

Q19. Un mutex garantit :

- A. L'exécution parallèle
- B. L'exclusion mutuelle
- C. La communication IPC
- D. La planification temps réel

Q20. Si un processus tente de prendre un mutex déjà verrouillé :

- A. Il continue
- B. Il est bloqué
- C. Il termine
- D. Il devient prioritaire

Q21. Un sémaphore compteur permet :

- A. Une exclusion stricte
- B. L'accès à N ressources identiques
- C. La gestion mémoire
- D. La préemption CPU

Q22. L'opération P() sur un sémaphore :

- A. Incrémente le compteur
- B. Décrémenté et bloque si négatif
- C. Réveille un processus
- D. Libère une ressource

Q23. Dans l'exemple des imprimantes, lorsque le compteur vaut -1 :

- A. Une imprimante est libre
- B. Aucun processus n'attend
- C. Un processus est bloqué
- D. Le système est saturé définitivement

Q24. Un programme est :

- A. Une instance en exécution
- B. Un processus actif
- C. Un code statique sur disque
- D. Un thread

Q25. L'appel système `fork()` :

- A. Remplace l'espace mémoire
- B. Crée un doublon du processus
- C. Tue le parent
- D. Lance un thread

Q26. Après `fork()`, le retour dans le processus enfant est :

- A. -1
- B. PID du parent
- C. 0
- D. 1

Q27. L'appel `exec()` :

- A. Duplique le processus
- B. Remplace le code du processus
- C. Crée un PCB
- D. Bloque le parent

Q28. L'exécution séquentielle impose :

- A. Une synchronisation par mutex
- B. Une attente explicite via `wait()`
- C. Une exécution parallèle
- D. Un quantum fixe

Q29. Le symbole `&` sous Linux permet :

- A. Une exécution pipe
- B. Une exécution en arrière-plan
- C. Une synchronisation
- D. Un verrouillage

Q30. Une exécution pipe est :

- A. Bidirectionnelle
- B. FIFO unidirectionnelle
- C. Préemptive
- D. Basée sur des mutex

Q31. Sous Linux, l'ordonnancement est basé principalement sur :

- A. Les processus
- B. Les jobs
- C. Les threads
- D. Les signaux

Q32. L'algorithme FCFS est :

- A. Préemptif
- B. Basé sur la priorité
- C. Non préemptif
- D. Circulaire

Q33. L'inconvénient majeur du FCFS est :

- A. Trop de commutations
- B. Famine des processus courts
- C. Favoriser les processus longs
- D. Forte préemption

Q34. SJF minimise principalement :

- A. Le temps de réponse maximal
- B. Le temps moyen d'attente
- C. Le nombre de processus
- D. La mémoire utilisée

Q35. SRT est :

- A. Non préemptif
- B. Identique à FCFS
- C. La version préemptive de SJF
- D. Basé sur le quantum

Q36. Le quantum typique du Round Robin est :

- A. 1–5 ms
- B. 5–10 ms
- C. 20–50 ms
- D. 100–200 ms

Q37. Un quantum trop court entraîne :

- A. Moins de commutations
- B. Trop de changements de contexte
- C. Plus de famine
- D. Moins de réactivité

Q38. Dans l'ordonnancement par priorité, une priorité élevée signifie :

- A. Exécution plus tardive
- B. Accès plus fréquent au CPU
- C. Blocage systématique
- D. Temps d'attente plus long

Q39. Avec un quantum de 50 ms, un processus utilisant 1 ms obtient une priorité :

- A. 1
- B. 2
- C. 25
- D. 50

Q40. L'objectif principal de la synchronisation est :

- A. Accélérer le CPU
- B. Éviter les incohérences de données
- C. Réduire la mémoire
- D. Supprimer la concurrence